Formation Data Engineer — RNCP 40167

TradeCorp Data Platform

Jalon 3 — Automatisation avec Apache Airflow

JALON 3 — Automatisation avec Apache Airflow

Contexte

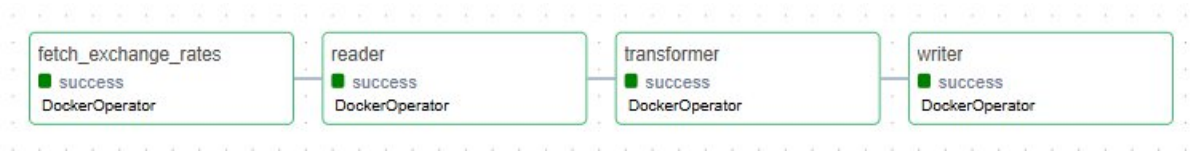
Votre pipeline TradeCorp est terminé et fonctionne : quatre scripts, testés, versionnés, qui vont chercher les taux de change, lisent les CSV dans le Data Lake, les transforment avec Spark, les enrichissent, et déposent le résultat en Parquet dans la zone clean. Il ne reste qu'un problème — et il est de taille : pour qu'il tourne, quelqu'un doit se connecter chaque matin et lancer quatre commandes à la main, dans le bon ordre.

Le DSI de TradeCorp est clair : la promesse initiale était « zéro intervention humaine ». Les fichiers CSV arrivent chaque nuit ; le pipeline doit tourner chaque nuit à 6h, tout seul, et l'équipe doit pouvoir voir d'un coup d'œil, au réveil, si tout s'est bien passé — et sinon, quelle étape a échoué et pourquoi.

C'est le rôle d'un orchestrateur. Vous allez ajouter Apache Airflow à votre docker-compose, puis décrire votre pipeline sous forme de DAG : quatre tâches, une par script, enchaînées dans l'ordre. Airflow se chargera de les lancer à l'heure, de réessayer en cas d'échec, de tout tracer.

> ⚠ *Vous n'écrirez aucune nouvelle logique de transformation dans ce jalon. Si le Jalon 2 Partie 2 est terminé et fonctionnel, le Jalon 3 consiste uniquement à orchestrer ce qui existe déjà.*

Ce que vous allez construire



Chaque tâche du DAG utilise un DockerOperator. Concrètement, quand Airflow exécute une tâche, il crée un nouveau conteneur à partir de l'image tradecorp-spark, y lance la commande (par exemple spark-submit reader.py), attend la fin, puis supprime le conteneur. Le conteneur n'existe que le temps de l'exécution.

Pourquoi ce choix plutôt qu'un BashOperator ou un PythonOperator ? Ces deux opérateurs s'exécutent directement dans le conteneur Airflow. Or Spark et Java ne sont pas installés dans ce conteneur (ils sont dans tradecorp-spark). Un spark-submit lancé depuis Airflow répondrait donc command not found. Seul le DockerOperator permet de lancer la commande dans le bon environnement.

Prérequis

- Jalon 2 Partie 2 validé : spark-submit src/pipeline.py tourne sans erreur de bout en bout
- Veille Airflow réalisée et présentée
- Cours Airflow lu (en particulier les sections Opérateurs et DAG TradeCorp)
- Le port 8080 disponible sur votre machine (ou choisir un autre port côté hôte)

7.1 — Ajouter Airflow à l'infrastructure

Q56 — Créer Dockerfile.airflow

Créer un Dockerfile.airflow basé sur apache/airflow:2.8.0 qui installe les dépendances depuis requirements.txt.
Ajouter apache-airflow-providers-docker à requirements.txt (c'est ce provider qui fournit le DockerOperator).
Installer les dépendances dans le Dockerfile.airflow

Q57 — Ajouter les services au docker-compose.yml

Ajouter deux services :
_ postgres (base de métadonnées Airflow => image postgres, avec un volume nommé pour la persistance)
_ Airflow (
    _ build depuis Dockerfile.airflow,
    _ port 8080 pour le webserver,
    _ volumes dags/, src/, .env, et surtout le socket Docker /var/run/docker.sock).
    La commande de démarrage doit :
    _ initialiser la base,
    _ créer un utilisateur admin mdp admin,
    _ lancer webserver et scheduler.

Indice : Le socket Docker est indispensable : sans lui, Airflow ne peut pas demander à Docker de créer des conteneurs. .

Q58 — Démarrer et vérifier

Lancer docker compose up -d --build. Suivre les logs (docker logs -f tradecorp_airflow) jusqu'à voir le webserver écouter sur le port 8080. Se connecter à http://localhost:8080 avec l'utilisateur créé.

> *✓ Validation 7.1 : l'interface Airflow est accessible, vide (aucun DAG, LOAD_EXAMPLES à false), et docker compose ps montre les 3 services (spark, postgres, airflow) en running.*

7.2 — Écrire le DAG

> *Le fichier va dans le dossier dags/ sur votre machine (celui monté dans le conteneur Airflow). Airflow le détecte automatiquement (pas besoin de redémarrer).*

Q59 — Le squelette du DAG

Créer dags/tradecorp_pipeline.py avec un DAG nommé tradecorp_etl_pipeline : start_date au 1er janvier 2024, exécution quotidienne à 6h (expression cron), catchup désactivé, 1 retry avec 5 minutes de délai, tags tradecorp / etl / spark. (Ne pas encore mettre de tâche) sauvegarder et vérifier que le DAG apparaît dans l'interface.

Indice : Un DAG sans tâche est valide et apparaît dans l'interface c'est le meilleur moyen de vérifier le squelette avant d'ajouter la complexité des tâches.

Q60 — Les montages de volumes

Le DockerOperator crée, à chaque exécution de tâche, un conteneur tout neuf à partir de l'image tradecorp-spark; un conteneur qui ne connaît que ce qu'il y a dans cette image (Python, Spark, Java, vos dépendances), mais qui ne voit rien de votre dossier de projet. Sans intervention, votre commande spark-submit /home/jovyan/src/reader.py échouerait avec "fichier introuvable", même si ce fichier existe très bien sur votre machine.
Pour résoudre ça, définissez la liste MOUNTS avec docker.types.Mount : trois bind mounts (src/, data/, .env) depuis le chemin absolu du projet sur votre machine vers /home/jovyan/... dans le conteneur. Sur Docker Desktop Windows, le chemin hôte s'écrit //c/Users/... (double slash, pas de deux-points).

NB : Le conteneur éphémère lancé par Airflow ne voit rien de votre projet tant que ces volumes ne sont pas montés explicitement dans la tâche.

Q61 — Les 4 tâches DockerOperator

Créer les 3 tâches. Toutes utilisent :
_ l'image tradecorp-spark,
_ docker_url unix://var/run/docker.sock,
_ network_mode tradecorp_default,
_ auto_remove,
_ mount_tmp_dir=False, et les mounts de la Q60.
_ indiquer la commande e exécuter pour lancer les jobs spark

Q62 — L'ordre d'exécution

Chaîner les tâches : fetch_exchange_rates → reader → transformer → writer. Vérifier dans l'onglet Graph de l'interface que les 4 tâches apparaissent bien reliées dans cet ordre.

✓ Validation 7.2 : le DAG est visible dans l'interface avec ses 4 tâches enchaînées, sans erreur d'import.

7.3 — Exécuter, observer, corriger

Q63 — Premier déclenchement manuel

Activer le DAG, puis le déclencher manuellement (bouton ▶ Trigger DAG). Suivre l'exécution dans l'onglet Grid : chaque tâche doit passer par queued → running → success.

Q64 — Lire les logs

Pour chaque tâche, ouvrir les logs depuis l'interface. Retrouver dans les logs de fetch_exchange_rates le nombre de devises récupérées ; dans ceux de writer le message d'upload vers ADLS. Copier ces deux extraits dans le README.

Q65 — Diagnostiquer un échec

Provoquer volontairement un échec :
_ renommer temporairement le fichier country_currency.csv dans ADLS (ou changer le nom dans le .env), déclencher le DAG, observer quelle tâche échoue et lesquelles passent en upstream_failed.
_ Lire le message d'erreur dans les logs.
_ Remettre le fichier en place, puis relancer uniquement la tâche échouée avec Clear et vérifier que la suite se relance d'elle-même.

Q66 — Vérifier l'idempotence

Déclencher le DAG deux fois de suite.
_ Vérifier ensuite dans ADLS que le dossier clean/orders_enriched/ ne contient qu'un seul fichier Parquet et pas deux. Si vous en trouvez deux, votre writer.py ne supprime pas l'ancien contenu avant d'uploader : corrigez-le.

Attention : Un pipeline orchestré tourne tous les jours. S'il accumule un fichier par exécution, au bout d'un mois le Data Lake contient 30 versions mélangées — et Spark les lira toutes ensemble.

Q67 — Le planning automatique

Vérifier dans l'interface la date/heure de la prochaine exécution planifiée (Next Run). Expliquer dans le README pourquoi, avec start_date en 2024 et catchup=False, Airflow n'a pas rejoué toutes les exécutions passées au moment de l'activation.

✓ VALIDATION JALON 3 : le DAG s'exécute de bout en bout depuis Airflow sans intervention sur les conteneurs Spark, un échec provoqué est correctement diagnostiqué et relancé, et deux exécutions consécutives ne produisent qu'un seul fichier Parquet dans la zone clean.

7.4 — Bonus : attendre l'arrivée des fichiers avec un FileSensor

Dans la vraie vie, le script legacy de TradeCorp dépose ses CSV à une heure variable dans la nuit. Lancer le pipeline à 6h « en espérant que les fichiers soient là » est fragile. Un sensor permet d'attendre un signal concret avant de démarrer.

Q68 (bonus) — Monter un dossier de déclenchement

Créer un dossier data/trigger/ dans le projet et le monter dans le conteneur Airflow (par exemple sur /opt/airflow/data/trigger). Le FileSensor s'exécute dans Airflow, il doit donc voir ce dossier depuis là.

Q69 (bonus) — Ajouter le sensor en tête de DAG

Ajouter une tâche FileSensor qui attend l'existence du fichier /opt/airflow/data/trigger/go.txt, vérifie toutes les 30 secondes, et abandonne après 1 heure. La placer avant fetch_exchange_rates dans la chaîne.

Indice : from airflow.sensors.filesystem import FileSensor — paramètres filepath, poke_interval, timeout.

Q70 (bonus) — Tester le déclenchement

Déclencher le DAG :
_ le sensor doit rester en running (il attend).
_ Créer alors le fichier go.txt dans data/trigger/ depuis votre machine.

_ Observer le sensor passer en success et la chaîne se lancer.
_ Supprimer go.txt à la fin

Livrables du Jalon 3

- Dockerfile.airflow et docker-compose.yml mis à jour (3 services)
- dags/tradecorp_pipeline.py — le DAG à 4 tâches
- Capture d'écran : vue Graph du DAG avec les 4 tâches reliées
- Capture d'écran : vue Grid montrant au moins un run complet en success
- Capture d'écran : les logs de la tâche writer avec le message d'upload ADLS
- Capture d'écran : le run avec l'échec provoqué (Q65) montrant failed et upstream_failed
- README mis à jour : comment lancer l'infrastructure, comment déclencher/relancer le DAG, extraits de logs, explication du catchup
- Bonus : capture du FileSensor en attente puis en success